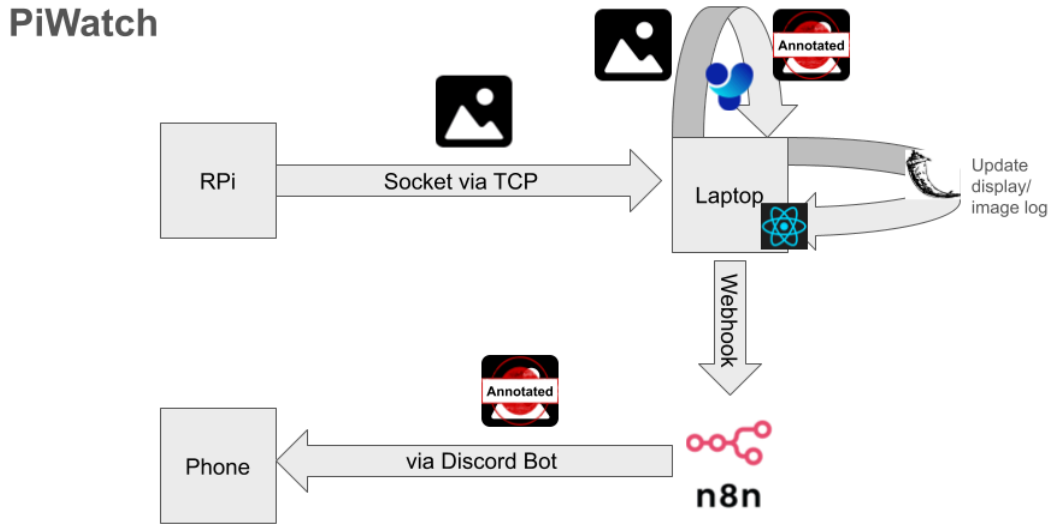
Team Members
Alex Frederic
Peyton Crawford

A short description of your project and its functionality
PiWatch is a security alert system that detects intruders then alerts your phone. The RPi has a camera that takes photos every 3–4 seconds then sends the image to the laptop. The laptop displays the image via a React frontend. If there is a human detected, you are alerted via Discord and the annotated image is stored locally and accessible through the frontend.

A block diagram that shows interaction between components



Briefly describes components, protocols used, key processing techniques, etc. along with important implementation/design choices.
We used the RPi Camera Module 2 connected to an RPi 4B via the camera ribbon port to capture images. These images were restricted to 640x480 pixel .jpg files to maintain consistency among the images and keep the file size to be transferred (about 56 KB) down while maintaining decent image quality for YOLO analysis. Each photo was then serialized and sent in 4-KB chunks to the laptop, where they would be run through the YOLOv8n model, the smallest of the YOLOv8 models. We chose this model because it was just recent enough to be sufficiently compatible with the rest of our code and accurate enough for our purposes, while lightweight and fast enough to prevent unnecessary slow-down between processing images or overwhelming the memory space of the program. Since TCP doesn't drop packets, we realized that without a delay function in piwatch.py the computer would be backed up due to processing of the image (through recombining chunks [~2.9s] and YOLO [~0.1s]). When YOLO is done, it saves both the original and annotated images as files, overwriting the ones taken previously. The program on the laptop searches the YOLO annotations to find how many humans it detects,

and if that number is greater than 0, it saves the annotated image again with the current time to log it and makes a POST request to our n8n workflow to message the user via Discord. We chose to save images this way so that the most recent image was always available to the user without overloading the drive with unnecessary images. It only saves the important ones showing intruders that can be accessed after the fact by the user via the frontend (labeled with times so they know exactly when they were taken). The receiving program is connected to a Flask server also running on the laptop to ensure the frontend stays up to date. It makes POST requests to the server whenever an intruder is detected to update a list of image paths containing intruders saved as a JSON file. The React frontend polls this server every second to keep the list of image paths up to date.

Alex did not use AI at all through this process. Peyton used AI significantly through the brainstorming process and for understanding the workflows. When we had significant front end issues, Peyton attempted to use AI to debug the issues but was mostly unsuccessful (except the image log).

Reflection on limitations and their causes as well as lessons learned.
Data transfer between the VM and any outside node was extremely slow due to the bridged connection. We ultimately avoided this by downloading all of the necessary modules to the host OS and ran it there (Windows). We would have liked to have avoided this since messing with the host OS typically has repercussions, but this was the only way to solve the speed issue (in addition to some other frontend issues discussed later). This would have been a good use case for Docker if we understood it well enough (or had more time). We also would have preferred to use a faster link between the RPi and VM, but that was beyond the scope of our understanding. There were many tools we looked into and tested but ultimately did not make it into our project including but not limited to: tkinter as a display, Supabase as a cloud storage platform, GCP for connecting Gmail, setting up SMTP for messaging, many different Yolo models, and more. We implemented MQTT but found it to be extremely slow. We then switched to Socket and it was still slow but we kept it since the messages came through more reliably than MQTT. This was because the Socket server directly sent the images from the RPi to the VM without passing through any middle-men brokers on the web (we were using test.mosquitto.org). The reason Socket was still slow was because of the bridged connection between the RPi and the VM (described above). This was fixed when we moved to Windows.
We wanted to run the YOLO on the RPi to reduce the data transfer so that it would only send anything to the VM iff a human was detected but because of the heaviness of ultralytics and open-cv we found this to be impossible without changing the RPi's OS. We should have planned more carefully and tested whether or not YOLO could run on the RPi before making it a pivotal part of our project and then having to quickly pivot to running it on the VM, and ultimately on Windows.
We wanted to show the most recently taken image in the React frontend and dynamically update it as images were taken. However, we failed to find a way to do this in this time frame. We struggled because we developed the React app on the local Windows machine but didn't test on the VM until mostly complete, at which point, we struggled to get it to work since NodeJS and npm aren't designed to work as seamlessly with Linux as with Windows. We did manage to

run it by building the project and using the LiveServer VSCode extension to run the dist files as a web server. However, to do this, we had to refactor where the images were placed by the backend, and it caused our HTTP requests to fail often. We ultimately had to resort back to running it in developer mode on the host OS to get it to work. The lesson here is to plan for your production environment, test your code in the most relevant places first, and use technologies that port well from device to device (i.e. using plain HTML/CSS/JS would likely have worked better on Linux than React).
We implemented sending images via n8n but for some reason it was extremely finicky and stopped working once we switched to the laptop. We aren't completely sure why this stopped working because when we tested the Webhook directly it worked but we eventually realized we simply didn't save the edits to the workflow we made. Note to selves: always save changes🙃.

What's special about our project
Our project allows the camera (connected via the small RPi) to be moved away from the bulky, visible laptop and hidden in sight of a sensitive location where would-be intruders would be unaware of its presence. Then, when it sends the images to the laptop for processing, the laptop can alert you via Discord anywhere in the world, letting you know within seconds whether any intruders are present. The distributed nature of the system allows for clandestine scanning to be made and long-range action to be taken while operating fully autonomously.

Youtube Link: https://youtube.com/shorts/W8BkkR_2Nfc?feature=share